

String concatenation using ||

- **Ref**

<https://www.sqlservercentral.com/articles/t-sql-in-sql-server-2025-concatenating-strings>

<https://drsql.link/2025/10/15/concatenating-values-as-character-data-including-in-sql-server-2025/>

- **Intro**

The `||` pipes operator in a string expression concatenates two or more character or binary strings, columns, or a combination of strings and column names into one expression (a string operator). For example, `SELECT 'SQL ' || 'Server';` returns `SQL Server`. The `||` operator follows the ANSI SQL standard for concatenating strings. In SQL Server you can also do string concatenation using `+` operator and the **CONCAT()** function.

- **Syntax**

expression || expression

- **Arguments**

expression

Any valid expression of any one of the data types in the character and binary data type category, except the xml, json, image, ntext, or text data types. Both expressions must be of the same data type, or one expression must be able to be implicitly converted to the data type of the other expression.

- **Remarks**

1. Some benefits of using the ANSI string concatenation syntax include:

- **Portability:** Using the ANSI standard `||` operator for string concatenation ensures that your SQL code is portable across different database systems. This means that you can run the same SQL queries on various database platforms without modification.
- **Consistency:** Adhering to the ANSI standard promotes consistency in your SQL code, making it easier to read and maintain, especially when working in environments with multiple database systems.
- **Interoperability:** The ANSI standard is widely recognized and supported by most SQL-compliant database systems, enhancing interoperability between different systems and tools.

- **Azure:** The || string concatenation operator is available in Azure SQL Managed Instance with the SQL Server 2025 or Always-up-to-date update policy.

2. String truncation behavior

If the result of the concatenation of strings exceeds the limit of 8,000 bytes, the result is truncated.

3. Zero-length strings and characters

The || operator behaves differently when it works with an empty, zero-length string than when it works with NULL, or unknown values.

4. Concatenation of NULL values

A string concatenation operation performed with a NULL value should also produce a NULL result. **The || operator does not honor the SET CONCAT_NULL_YIELDS_NULL option, and always behaves as if the ANSI SQL behavior is enabled, yielding NULL if any of the inputs is NULL. This is the primary difference in behavior between the + and || concatenation operators.**

- **Use cases**

1. Literal String

```
SELECT
  'Denver ' || 'Broncos' AS Spacein1stString,
  'Denver ' || 'Broncos' AS Spacein2ndString,
  'Denver' || ' ' || 'Broncos' AS ThreeStrings
```

2. String + String from columns

```
USE [AdventureWorks2022]
GO

SELECT (LastName || ', ' || FirstName) AS Name
FROM Person.Person
ORDER BY LastName ASC, FirstName ASC;
```

3. String + Date

```
USE [AdventureWorks2022]
GO

SELECT 'The order is due on ' || CONVERT(VARCHAR(12), DueDate, 101)
```

```

FROM Sales.SalesOrderHeader
WHERE SalesOrderID = 50001;
GO

```

4. Multiple Strings

```

USE [AdventureWorks2022]
GO

SELECT
    (LastName || ', ' + SPACE(1) || SUBSTRING(FirstName, 1, 1)
    || '. ') AS Name
    , e.JobTitle
FROM Person.Person AS p
JOIN HumanResources.Employee AS e
    ON p.BusinessEntityID = e.BusinessEntityID
WHERE e.JobTitle LIKE 'Vice%'
ORDER BY LastName ASC;
GO

```

5. Large Strings

```

DECLARE @x VARCHAR(8000) = REPLICATE('x', 8000);
DECLARE @y VARCHAR(MAX) = REPLICATE('y', 8000);
DECLARE @z VARCHAR(8000) = REPLICATE('z', 8000);

SET @y = @x || @z || @y;

-- The result of following select is 16000
SELECT LEN(@y) AS y;
GO

```

6. Var + String

```

DECLARE @player VARCHAR(20) = 'Bo Nix ', @num VARCHAR(2) = '10'

SELECT
    @player || @num AS 'Var || Var'
    , @player || '- 10' AS 'Var || String'
    , 'Bo - ' || @num AS 'String || Var'

```

7. Var + conversions + ||

```

/* Example I */
DECLARE @player VARCHAR(20) = 'Bo Nix',
        @num      VARCHAR(2)  = '10',
        @position VARCHAR(2)  = 'QB';

SELECT @player || ' - ' || @num || '-' || @position
UNION
SELECT @player || ' - ' || 10 || ' - ' || @position
UNION
SELECT @player || ' - ' || @num || '-' || @position ||
      ' Last Start:' || GETDATE()
UNION
SELECT @player || ' - ' || @num || '-' || @position ||
      ' Next Start:' || CAST('2025-10-24' AS DATE)

/* Example I */
DECLARE @player VARCHAR(20) = 'Bo Nix',
        @num      VARCHAR(2)  = '10',
        @position VARCHAR(2)  = 'QB'

SELECT @player || ' - ' || @num || ' - ' || @position || ' CMP%:'
      || 64.6 || ' DoB:' || CAST('2000/2/25' AS DATE)

```

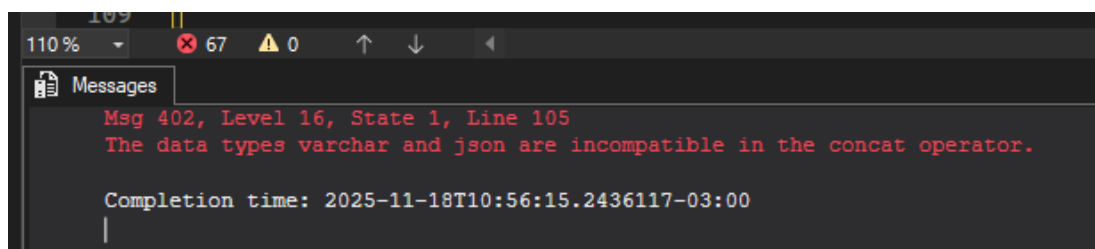
8. || + JSON

```

DECLARE @player VARCHAR(20) = 'Bo Nix',
        @num      VARCHAR(2)  = '10',
        @position VARCHAR(2)  = 'QB',
        @stats    JSON        = '{"PassYards":1277,"TD":9,"INT":4}'

SELECT @player || ' - ' || @num || ' - ' || @position || ' CMP%:'
      || 64.6 || ' - ' || @stats

```



9. Binary + Binary

```

DECLARE @b VARBINARY(100) = CAST('Next Opponent:' AS VARBINARY(100))
        , @o VARBINARY(100) = CAST('Giants' AS VARBINARY(100)) ;

SELECT @b || @o

SELECT CAST(@b || @o AS VARCHAR(100))

```

10. Binary + String + Binary

```

/* Example I */
DECLARE @b VARBINARY(100) = CAST('Next Opponent:' AS VARBINARY(100))
        , @o VARBINARY(100) = CAST('Giants' AS VARBINARY(100)) ;

SELECT CAST(@b || ' ' || @o AS VARCHAR(100))

/* Example II */
DECLARE @b VARBINARY(100) = CAST('Next Opponent:' AS VARBINARY(100))
        , @o VARBINARY(100) = CAST('Giants' AS VARBINARY(100)) ;

SELECT CAST(@b || CAST(' ' AS VARBINARY(100)) || @o AS VARCHAR(100))

```

11. || + NULL

```

DECLARE @player VARCHAR(20) = 'Bo Nix',
        @num VARCHAR(2) = '10',
        @position VARCHAR(2)

SELECT @player || ' - ' || @num || ' - ' || @position ||
        '-2025 Season'

```

12. Use of CAST and CONVERT

```

/* EXAMPLE I
   No CONVERT or CAST function is required because this example
   concatenates two binary strings. */
DECLARE @mybin1 VARBINARY(5), @mybin2 VARBINARY(5);

SET @mybin1 = 0xFF;
SET @mybin2 = 0xA5;

SELECT @mybin1 || @mybin2;

```

```

/* EXAMPLE II
   A CONVERT or CAST function is required because this example
   concatenates two binary strings plus a space. */
DECLARE @mybin1 VARBINARY(5), @mybin2 VARBINARY(5);

SET @mybin1 = 0xFF;
SET @mybin2 = 0xA5;

SELECT @mybin1 || ' ' || @mybin2;

/* CONVERT */
SELECT CONVERT(VARCHAR(5), @mybin1) || ' ' ||
        CONVERT(VARCHAR(5), @mybin2);

/* Using CAST */
SELECT CAST(@mybin1 AS VARCHAR(5)) || ' ' ||
        CAST(@mybin2 AS VARCHAR(5));

```

13. Assigning Values to variables (||= as +=)

```

DECLARE @v1 varchar(10) = 'a'
SET @v1 ||= 'b';
SELECT @v1

DECLARE @v2 varbinary(10) = 0x1a;
SET @v2 ||= 0x2b;
select @v2;

```

• String Concatenation Operator Comparison

Operator	Advantages	Disadvantages	When to Use
"+"	Familiarity: Widely known and used by veteran SQL Server developers Syntax: Shorter and more concise than a function call	NULL Handling: If any operand is NULL, the entire result is NULL (by default). Data Type Handling: Requires explicit CAST() or CONVERT() when mixing strings with non-string data types (like INT or DATE). Failure to	When maintaining legacy code where its NULL behavior is already managed. For simple concatenation of two variables/columns that are known to be strings and non-NULL.

		cast can lead to errors or mathematical addition instead of concatenation.	
"CONCAT()"	NULL Handling: Automatically treats NULL inputs as an empty string ("), preventing the entire result from becoming NULL. Data Type Handling: Implicitly converts all arguments (e.g., INT, DATE, FLOAT) to string types before concatenation, eliminating the need for explicit CAST/CONVERT. Versatility: Accepts a variable number of arguments (2 to 254).	Function Overhead: As a function, it may have slightly more overhead than an operator, though performance differences are usually minimal. Verbosity: The syntax is longer than using an operator.	When you need to combine strings with mixed data types (numbers, dates, etc.) without explicit casting. When you want NULL values to be automatically handled as empty strings (") to ensure a non-NULL result.
	ANSI Standard: Aligns SQL Server with the ANSI standard, improving code portability across other major database platforms (like Oracle or PostgreSQL). Operator: Shorter and more concise syntax than a function call. Data Type Handling: Implicitly converts most data types to a string for concatenation (similar to CONCAT())	NULL Handling: If any operand is NULL, the entire result is NULL (This is the strict ANSI standard behavior). New: Developers new to SQL Server 2025 might be less familiar with it initially.	To follow the ANSI SQL standard for all new development. When combining strings with other data types where you still need to respect the ANSI-standard NULL propagation